Universidade Federal do Paraná (UFPR)
Setor de Ciências Exatas – Curso de Ciência da Computação

Trabalho Prático de Redes de Computadores II – Turma 2026/2

Antonio da Ressurreição Filho e Nathan Endo

Professor: Prof. Dr. Elias Procópio Duarte Jr.

Curitiba
2026

Dias

Dia 1 – 20 de outubro de 2026 **2**

Dia 2 – 28 de outubro de 2026 **3**

Dia 3 – 01 de novembro de 2026 **5**

Dia 4 – 3 de novembro de 2026 **6**

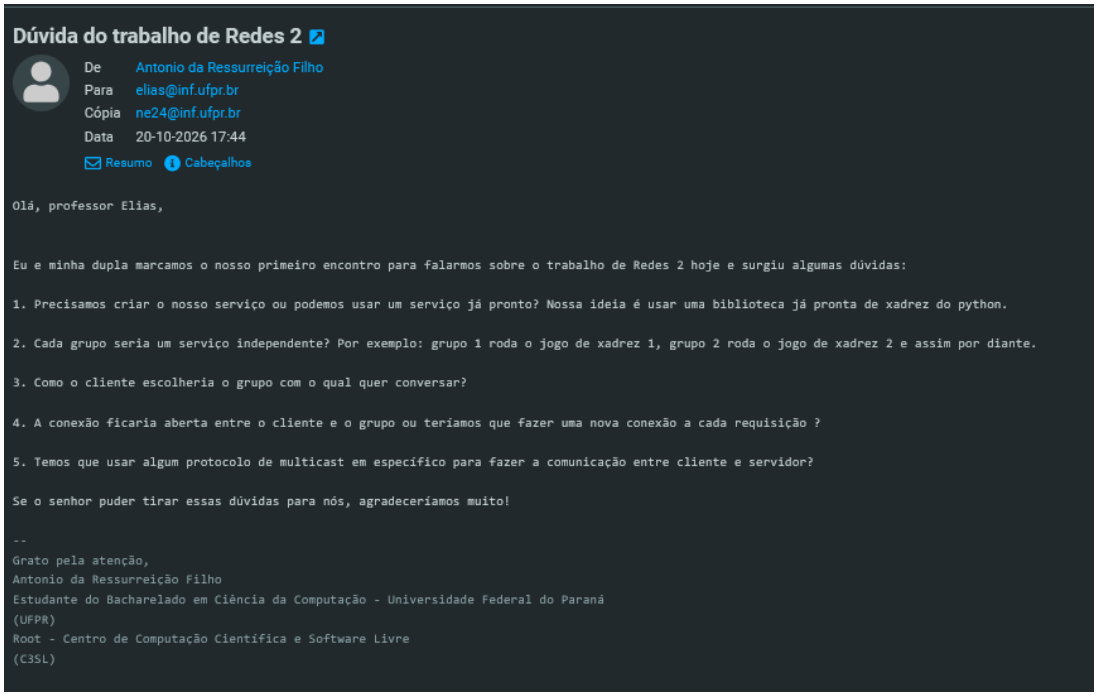
Dia 5 – 5 de novembro de 2026 **8**

Dia 1 – 20 de outubro de 2026

Após as orientações passadas pelo professor na semana do dia 12/10/2026 a 16/10/2026 sobre o trabalho, nós começamos montando a nossa equipe e planejando o nosso primeiro encontro hoje, dia de 20/10/2026.

O nosso primeiro desafio do dia foi tentar achar o serviço perfeito para se encaixar no nosso programa. Depois de algumas discussões, decidimos o serviço que mais se encaixaria conosco e com o nosso programa: um serviço de **xadrez** provido pela biblioteca **chess** do Python. Com isso, ficamos na dúvida se o professor nos deixaria implementar esse serviço mais complexo porém "pronto"da biblioteca. **Comentário vindo do futuro:** descobrimos que isso seria "over"demais, porém encaramos o desafio e topamos usar esse serviço de xadrez, com a devida permissão do professor!

Ademais, antes de finalmente implementar o código, precisamos planejar os nossos próximos passos do trabalho e fazermos a divisão certinha das funções de cada um de nós. Dessa forma, várias dúvidas surgiram e não sabíamos exatamente como respondê-las. Por esse motivo, mandamos um email para o professor Elias tirando todas as nossas dúvidas:



Dúvida do trabalho de Redes 2

De Antonio da Ressurreição Filho
Para elias@inf.ufpr.br
Cópia ne24@inf.ufpr.br
Data 20-10-2026 17:44
Resumo Cabeçalhos

Olá, professor Elias,

Eu e minha dupla marcamos o nosso primeiro encontro para falarmos sobre o trabalho de Redes 2 hoje e surgiu algumas dúvidas:

1. Precisamos criar o nosso serviço ou podemos usar um serviço já pronto? Nossa ideia é usar uma biblioteca já pronta de xadrez do python.

2. Cada grupo seria um serviço independente? Por exemplo: grupo 1 roda o jogo de xadrez 1, grupo 2 roda o jogo de xadrez 2 e assim por diante.

3. Como o cliente escolheria o grupo com o qual quer conversar?

4. A conexão ficaria aberta entre o cliente e o grupo ou teríamos que fazer uma nova conexão a cada requisição ?

5. Temos que usar algum protocolo de multicast em específico para fazer a comunicação entre cliente e servidor?

Se o senhor puder tirar essas dúvidas para nós, agradeceríamos muito!

--
Grato pela atenção,
Antonio da Ressurreição Filho
Estudante do Bacharelado em Ciência da Computação - Universidade Federal do Paraná
(UFPR)
Root - Centro de Computação Científica e Software Livre
(C3SL)

Informações vindas do futuro: Inesperadamente deu um erro no email enviado nesse dia para o professor e ele foi parar na caixa de spam, não tendo sido respondido infelizmente. Ao longo dos outros dias, muitas dessas dúvidas foram respondidas:

1. O professor respondeu essa dúvida alguns dias depois.
2. Entendemos que todos os servidores ou fazem parte do grupo do Multicast ou estão mortos.
3. O grupo é o Multicast.

4. Essa pergunta foi feita de forma equivocada e não tínhamos entendido direito um conceito em específico, depois entendemos que o programa irá ficar rodando.

5. O único protocolo que vamos usar é o UDP (User Datagram Protocol), já que se trata de comunicação multicast.

Dia 2 – 28 de outubro de 2026

Neste segundo encontro, com a cabeça um pouco mais fria e o caminho mais claro, sentamos para "bater o martelo"e estruturar de vez a arquitetura do nosso projeto. A ideia foi transformar as discussões do primeiro dia em decisões concretas para podermos colocar a mão na massa no próximo encontro.

O entendimento do Projeto

Entendemos que o sistema seria composto por um único grupo de servidores trabalhando em equipe para manter a mesma aplicação viva (a nossa partida de xadrez). O nosso objetivo aqui não é criar um servidor robusto para hospedar várias partidas simultâneas, mas sim focar na tolerância a falhas: se um servidor cair, o jogo não pode parar de jeito nenhum.

Para organizar a casa, entre os servidores vivos, um vai assumir o papel de líder (o primário). É ele quem vai processar as requisições dos clientes. Os outros servidores vão atuar apenas como réplicas (*backups*), acompanhando a partida de camarote e mantendo uma cópia atualizada do estado do tabuleiro.

Comunicação

Para a comunicação, optamos por usar *sockets* UDP, pois é o protocolo necessário para multicast. Iremos configurar os servidores e clientes para trocarem mensagens pelo grupo *multicast*. A única exceção que abrimos foram as respostas diretas do líder para o cliente, que decidimos mandar em *unicast*.

Definimos um padrão próprio para o formato das mensagens: `TIPO|campo1|campo2|...`. Escolhemos o caractere `|` como separador por um motivo simples: a nossa representação do tabuleiro (FEN) já vem cheia de espaços, então usar um espaço como delimitador ia dar muita dor de cabeça no *parsing*. Todos os tipos de mensagem serão amarrados em uma enumeração (`Enum`) no código para manter tudo organizado.

A Lógica do Xadrez

Para a lógica do jogo, a biblioteca `python-chess` caiu como uma luva. Ela serializa o estado do tabuleiro no formato FEN (*Forsyth–Edwards Notation*), o que é maravilhoso porque resume a partida inteira em uma *string* curtinha, de menos de 100 caracteres.

Definimos o fluxo de interação assim:

- **Leitura:** Quando o cliente quer ver o jogo, ele manda um `READ` pro grupo. O líder pega essa requisição e responde direto para ele com um `STATE`, mandando o FEN e como está a situação do tabuleiro.

- **Escrita:** Para jogar, o cliente manda um `MOVE` com o lance em notação UCI (tipo `e2e4`). O líder atua como juiz e valida o lance; se for uma jogada ilegal, ele devolve um `ERRO`. Se for válida, ele aplica o lance, incrementa o nosso contador (número de sequência) e "grita"um `SYNC` para o grupo com o novo FEN. As réplicas ouvem, atualizam seus tabuleiros e nos respondem com um `ACK`. Só depois de receber essas confirmações é que o líder manda um `OK` pro cliente.

Para deixar a interface do cliente mais rica, faremos o líder enviar também um texto descrevendo o que está rolando na partida: de quem é a vez, se está em xeque, xeque-mate, ou se deu empate (seja por afogamento ou falta de material).

a biblioteca `python-chess`[1]

Nossas Decisões de Projeto

- **Tamanho do *payload*:** Como o nosso FEN é minúsculo comparado ao MTU padrão da rede (que gira em torno de 1500 *bytes*), demos sorte: as mensagens de `SYNC` cabem tranquilamente em um único datagrama, sem precisarmos nos preocupar com fragmentação de pacotes.

- **Confirmação "na mão":** Como sabemos bem das aulas teóricas, o UDP não garante a entrega. Então, teremos que implementar a confirmação (`ACK`) por conta própria, direto na camada de aplicação. Usaremos o número de sequência do `SYNC` para não nos perdermos sobre qual pacote estaria sendo confirmado.

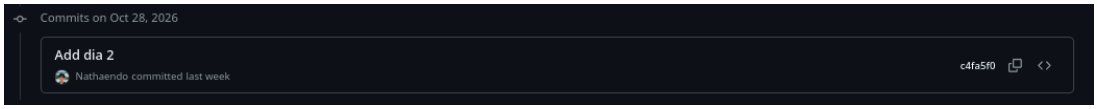
- **Não esperar para sempre (*Timeout*):** Colocaremos o líder para esperar os ACKs por, no máximo, 1 segundo. Vai que uma réplica morre logo depois de receber o `SYNC`? Se não tiver esse limite, o líder ficaria travado ali indefinidamente esperando uma resposta que nunca ia chegar.

- **Uma *Thread* por escrita:** Decidimos que cada `MOVE` será processado em uma *thread* própria. Se deixássemos a mesma *thread* que escuta a rede responsável por isso, ela ficaria presa esperando os ACKs e pararia de receber requisições novas (um clássico *deadlock*). Para não virar bagunça e as requisições se atropelarem, usaremos um *lock* para garantir que os lances sejam processados um de cada vez.

- **ACK no *multicast*:** Durante os nossos testes locais do dia 4, rodamos todos os servidores na mesma máquina, ou seja, todo mundo dividindo a mesma porta. Se a gente mandasse o ACK em *unicast*, corria o risco de ele ser entregue ao processo errado. A saída foi mandar o ACK para o grupo todo, e o líder que se vire para filtrar quem respondeu pelo número de sequência. O intuito era ser mais fácil análisar a corretude do código então não geramos logs, iremos tirar um dia especial para testar o código em máquinas diferentes e gerar os logs válidos para o projeto. **(Informações vindas do futuro)**

Todas as informações da nossa segunda reunião foram versionadas no repositório, não começamos a codar de fato no segundo dia, então foi só o README mesmo.

[1] https://python-chess.readthedocs.io



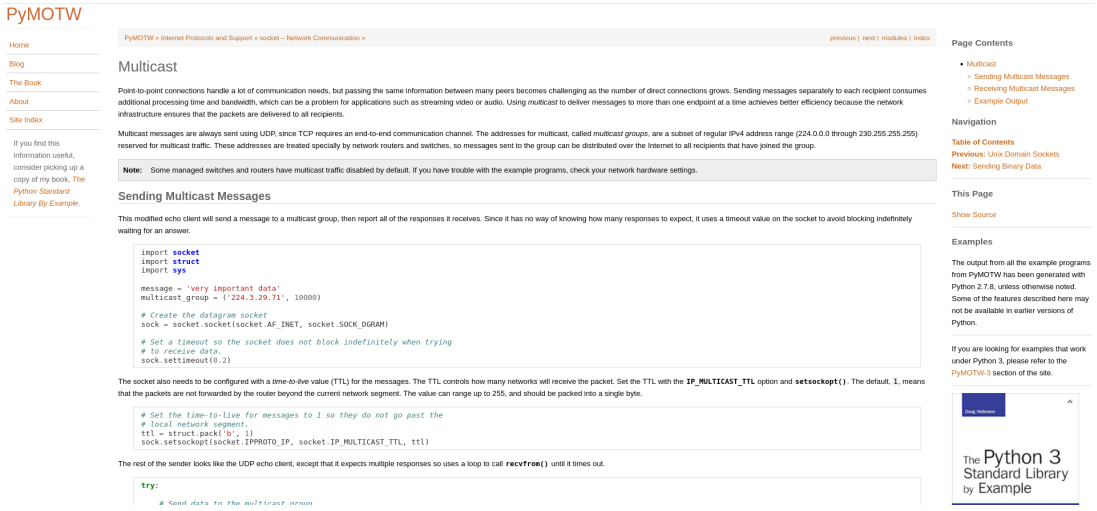
Dia 3 – 01 de novembro de 2026

Chegamos no primeiro dia de implementação, depois de todo o planejamento que tivemos nos outros encontros. Nossa ideia inicial no dia de hoje foi construir a comunicação cliente-servidor, estabelecer o heartbeat entre os servidores, a fim de que saibam quem ainda está no grupo e quem morreu, e por fim estabelecer a eleição do líder do grupo, o que foi estabelecido junto com o professor que será o servidor com o menor índice de UID.

Esqueleto da comunicação Multicast

Primeiramente, tivemos que aprender a como fazer o esqueleto básico da comunicação multicast do socket cliente-servidor na linguagem Python. Pesquisamos na internet as melhores formas de estabelecer essa comunicação e achamos um site que explicava perfeitamente e nos dava um molde de esqueleto bem simples para começarmos:

PyMOTW



A partir desse esqueleto e com as funções do Python para comunicação multicast com socket já estabelecidas, conseguimos fazer uma transmissão de dados entre cliente e servidores na própria máquina com êxito.

Heartbeat

Na segunda parte do dia fizemos uma comunicação heartbeat entre os servidores da rede a fim de todos terem a consciência de quais servidores ainda estão conectados na rede a cada 1 segundo, período de tempo estabelecido por nós. Tivemos que importar uma biblioteca específica do Python (**threading**) para ter duas ou mais funções executando ao mesmo tempo, isso é, tanto a comunicação entre cliente e servidores quanto o heartbeat dos servidores rodando juntos.

Para essa parte do projeto foram necessárias algumas funções e estrutura de dados extras. Em primeiro lugar, cada servidor deve ter uma tabela consigo guardando a informação de quem ainda está ativo na rede e quem não está mais. A estrutura de dados que escolhemos para isso foi um dicionário, em que funciona basicamente como uma estrutura de dados 'chave-valor'. Ademais, precisou-se de funções adicionais para guardar o valor dos UIDs de cada servidor nesse dicionários locais. Finalmente, implementamos uma última função extra para consultar o tempo do último heartbeat mandado pelo servidor e o tempo atual, com o intuito de remover da tabela local os servidores mortos.

Portanto, a lógica final do heartbeat foi chamar essas funções de pegar o UID, atualizar a tabela e conferir o tempo atual com o último sinal enviado em uma das threads rodando, chamada de background em nosso código.

Dia 4 – 3 de novembro de 2026

Neste quarto encontro, finalmente tiramos o xadrez do papel. Partimos do código que já tínhamos do encontro anterior (heartbeat e eleição de líder funcionando) e implementamos o fluxo de leitura e escrita que tínhamos desenhado no Dia 2. No fim do dia, já conseguíamos jogar uma partida inteira e derrubar o líder no meio dela sem o jogo parar.

Quem joga do outro lado?

Antes de codar, precisávamos decidir quem faria o lance de resposta. Consideramos três opções: o cliente controlar os dois lados, sortear um lance legal ou chamar o Stockfish. O Stockfish é fácil de chamar pelo `python-chess`, mas exige o binário instalado em toda máquina que pode virar líder, o que pode dar dor de cabeça nas máquinas do laboratório.

Como o foco do trabalho é a tolerância a falhas e não a inteligência do adversário, decidimos começar pelo mais simples: o cliente controla as brancas e as pretas. Deixamos o ponto de troca isolado na função `lance_adversario()`, que por enquanto não faz nada. Como as réplicas só recebem o FEN pronto, trocar isso depois não mexe em nada da replicação.

O que implementamos

No servidor, a regra foi não mexer no que já funcionava: o heartbeat e a eleição ficaram como estavam, e todo o xadrez entrou como acréscimo.

- **Protocolo:** criamos o `Enum` com os tipos de mensagem (`READ`, `MOVE`, `SYNC`, `ACK`, `STATE`, `OK` e `ERRO`) e uma função que monta as mensagens no formato `TIPO|campo1|campo2`.

- **Leitura e escrita:** implementamos o fluxo do Dia 2. O líder responde o `READ` com o tabuleiro; no `MOVE`, valida o lance, aplica, manda o `SYNC` e espera os `ACK` por até 1 segundo antes de responder `OK`. As réplicas atualizam o tabuleiro e confirmam. Um detalhe que apareceu na prática: o líder também recebe o próprio `SYNC` de volta pelo *loopback*, então ele precisa ignorar essa mensagem.
- **Textinhos da partida:** o líder manda junto com o tabuleiro de quem é a vez e se houve xeque, xeque-mate, afogamento ou empate por falta de material.
- **Blindagem da *thread* que escuta a rede:** colocamos o tratamento das mensagens de xadrez dentro de um `try/except`. Sem isso, uma mensagem malformada (um `MOVE` sem lance, por exemplo) derrubaria a *thread*, e o servidor continuaria mandando heartbeat, parecendo vivo, mas sem ouvir mais nada.

O cliente, que antes só mandava uma mensagem de teste, virou interativo: consulta o tabuleiro, mostra na tela, pede o lance e manda, repetindo até o fim da partida. Tivemos que ajustar duas coisas nele. O *buffer* de recepção era de 16 *bytes*, e o FEN não cabe nisso, então subimos para 1024. E o *timeout* de 0,2 segundo era menor que o tempo que o líder pode passar esperando os ACKs; se o cliente desistisse antes, a resposta atrasada ficaria no *buffer* e bagunçaria a requisição seguinte. Subimos para 2 segundos.

Testando

Testamos tudo em uma máquina só, com três servidores e o cliente rodando em terminais separados. Jogamos um mate do pastor e, depois do terceiro lance, matamos o servidor 1, que era o líder. Uns 3 segundos depois, os outros dois perceberam a falha pelo heartbeat e o servidor 2 assumiu. A partida continuou exatamente de onde estava, e o cliente nem percebeu a troca: só recebeu, no fim, o "Xeque-mate! Brancas venceram". Também conferimos que lances inválidos são recusados com `ERRO`.

Como comentamos no Dia 2, esses testes serviram para verificar se o código está certo, e não para gerar os logs oficiais. Esses ficam para um dia de testes em máquinas diferentes.

Perrengues de ambiente

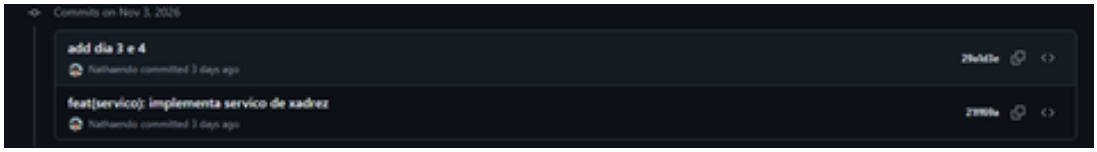
- **Instalação do `python-chess`:** o `pip install chess` foi bloqueado pelo sistema (erro *externally-managed-environment*, coisa do Debian/Ubuntu) e o pacote não existia no `apt`. A saída foi usar um ambiente virtual (`venv`), que de quebra não precisa de `sudo` e deve funcionar nas máquinas do laboratório. Também criamos um `requirements.txt` para facilitar a instalação.

O que ficou para depois

- Se um servidor com UID menor entra no meio da partida, ele vira líder com o tabuleiro zerado, porque ainda não existe transferência de estado na entrada.
- Se o líder cair bem entre aplicar o lance e as réplicas receberem o `SYNC`, o lance se perde.

- Avaliar a troca do ACK para *unicast* nos testes com máquinas diferentes.
- Colocar um adversário de verdade (lance aleatório ou Stockfish).

Todo o código do dia e a atualização do README foram versionados no repositório.



Dia 5 – 5 de novembro de 2026

Até aqui, todos os nossos testes tinham sido em *localhost*. Neste quinto encontro, colocamos o grupo para rodar de verdade, em três computadores da mesma rede local: o desktop com o UID 1, o servidor[2] com o UID 2 e o notebook com o UID 3, que também rodou o cliente. Guardamos os logs separados por cenário e as partidas jogadas, e eles estão disponíveis no site do projeto.

Primeiro problema: as máquinas não se enxergavam

Logo de cara, as máquinas não se enxergavam direito. O notebook não recebia mensagem de ninguém e se elegia líder sozinho, com o UID 3. O desktop parecia estar certo, mas só porque tem o menor UID: ele também estava sozinho, se elegendo por conta própria. Só percebemos isso porque não aparecia nenhum `Recebido de` no log dele.

No desktop, o culpado era o *firewall*, e liberar o UDP da rede local resolveu. A lição que ficou foi que não dá para olhar só para o líder: é preciso conferir o `[ESTADO] Vivos atuais` em cada uma das máquinas, porque um servidor isolado com o menor UID se comporta exatamente como um líder legítimo.

[2] O "servidor" é um computador do *homelab* do integrante Nathan Endo.

Cenários que testamos

Cenário	O que fizemos	Resultado
Heartbeat e eleição	Subimos os três servidores	Todos receberam os três heartbeats e elegeram o UID 1
Lances e replicação	Jogamos `e2e4`, `e7e5` e `g1f3`	O líder aplicou e as duas réplicas receberam, de `seq` 1 a 3
Lance inválido	Mandamos `e2e5` e `abc`	Os dois foram recusados, sem mudar o tabuleiro
Queda do líder	Derrubamos o desktop no meio da partida	O UID 2 assumiu e aceitou `g1f3` no mesmo tabuleiro
Queda de réplica	Derrubamos o UID 2 e logo mandamos `e7e5`	O líder avisou `Sem ACK de [2]` e respondeu `OK` mesmo assim
Líder que volta	O desktop caiu e voltou no meio da partida	Primeiro falhou; depois da correção, funcionou

O erro que os testes mostraram: o líder que voltava zerava a partida

O cenário do líder que volta revelou justamente a pendência que tínhamos deixado anotada no Dia 4. Quando o UID 1 voltou, ele virou líder de novo por ter o menor UID, só que com o tabuleiro inicial. O cliente mandou `b8c6`, levou um "Lance inválido" e o tabuleiro voltou para o começo.

Nossa primeira ideia foi mandar um `SYNC` sempre que o líder mudasse. O problema é que, até esse `SYNC` chegar, o servidor que acabou de subir já se acharia líder e poderia responder com o tabuleiro errado.

A solução que adotamos foi outra: quem sobe manda um `JOIN|uid` e só pode ser líder depois de receber o tabuleiro. Quem responde é o líder calculado sem contar com o servidor que está entrando (`elege_lider(ignorar=uid)`). Se ninguém responder em 3 segundos, é porque não há partida em andamento, e o servidor segue com o tabuleiro inicial.

Testamos de novo: o UID 1 recebeu o `seq=3` do líder 2, assumiu a liderança e aceitou o `b8c6` no tabuleiro certo. Uma réplica reiniciada também recebeu o tabuleiro corretamente. Depois da correção, refizemos todos os cenários; só o log `reentrada-lider-antes` é da versão antiga, e o guardamos justamente para mostrar o erro.

O efeito colateral é que, quando todos os servidores sobem juntos, o grupo leva 3 segundos para começar a responder, que é o tempo de espera pelo `JOIN`.

Partidas para ver as situações de jogo

Além dos cenários de falha, jogamos algumas partidas para conferir os textinhos de situação que o líder manda:

- **Mate do pastor:** "Xeque-mate! Brancas venceram".
- **Mate do tolo:** "Xeque-mate! Pretas venceram".
- **Afogamento:** "Empate por afogamento (stalemate)".

- **Lances especiais:** *en passant*, roque e promoção foram aceitos, e a promoção deu "Xeque! Vez das pretas".
- **Repetição:** o líder anunciou "Fim de jogo: 1/2-1/2", mas o cliente continuou pedindo lance. Isso acontece porque o FEN não guarda o histórico de repetições: o líder sabe que houve repetição, mas o cliente, que só enxerga o FEN, não.

Todo o código do dia, os logs e as partidas foram versionados no repositório e também estão disponíveis no site de apresentação do projeto.

